Modbus协议

简介

Modbus 是一种在工业自动化（PLC、仪表的数据采集）、能源管理（接入IoT平台）中广泛使用的通信协议，用于让不同设备之间进行数据交换。它由 Modicon公司（后来被施耐德电气收购了）于 1979 年提出，该协议的初衷是为了与其所生产的可编程逻辑控制器配套使用。凭借开放、灵活等特点，Modbus成为目前全球使用最广、兼容性最好、实施成本最低的工业协议之一。具有如下优点：

- Modbus协议标准开放、公开发表且无版权要求
- Modbus协议支持多种电气接口，包括RS232、RS485、TCP/IP等，还可以在各种介质上传输
- Modbus协议消息帧格式简单、紧凑、通俗易懂。用户理解和使用简单，厂商容易开发和集成，方便形成工业控制网络

通信方式

Modbus采用主从方式通信，不能同步进行通信，总线上每次只有一个数据进行传输，即主机发送，从机应答，主机不发送，总线上就没有数据通信。并且，Modbus并没有忙机制判断，比方说主机给从机发送命令，从机没有收到或者正在处理其他东西，这时候就不能响应主机，因为 modbus 的总线只是传输数据，没有其他仲裁机制，所以需要通过软件的方式来判断是否正常接收。

数据模型

Modbus 用 4 类 16 位寄存器划分数据，所有设备遵循相同地址规则：

寄存器类型	数据类型	读写	功能码
离散输入（DI）	1 位（布尔）	只读	02
线圈（Coil）	1 位（布尔）	读写	01、05、15
输入寄存器（IR）	16 位整数	只读	04
保持寄存器（HR）	16 位整数	读写	03、06、16

传输模式

Modbus协议的所有 16 位的参数（比如寄存器地址、读取数量），必须把 “高字节” 放前面，“低字节” 放后面（大端序）。但 RTU 帧末尾的 CRC 校验码（2字节，16位）是唯一例外，不按高字节在前，而是低字节在前、高字节在后。

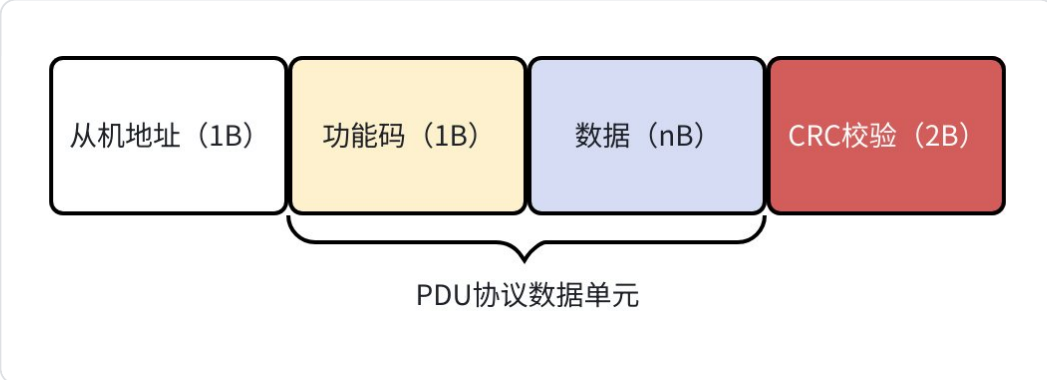
协议分为以下三种，Modbus-RTU、Modbus-ASCII、Modbus-TCP：

1. Modbus-RTU

Modbus-RTU（Remote Terminal Unit）是 Modbus 协议族中最常用的串行版本。它通过RS-485/RS-232串口传输数据，采用严格的帧结构和 CRC 校验保证可靠性。

帧结构如下图所示：采用二进制编码

帧结构 = 从机地址 (1B) + PDU（功能码 (1B) + 数据 (nB) ）+ CRC 校验 (2B)



从机地址: 每个从机都有唯一地址，占用一个字节，范围0-255，其中有效范围是1-247,其中255是广播地址。

功能码: 占用一个字节，功能码的意义就是, 说明指令的作用，比如你可以查询从机的数据，也可以修改从机的数据,所以不同功能码对应不同功能。

数据: 根据功能码不同，有不同功能，比方说功能码 `0x03` 是查询从机的数据，这里就是查询数据的地址和查询字节数等。

CRC校验：在数据传输过程中可能数据会发生错误，CRC检验检测接收的数据是否正确。

Modbus-ASCII 有开始字符和结束字符，可以作为一帧数据开始和结束的标志。而Modbus-RTU没有这样的标志，需要用时间间隔来判断一帧报文的开始和结束，协议规定的时间为3.5个字符周期，就是说一帧报文开始前，必须有大于3.5个字符周期的空闲时间，一帧报文结束后，也必须要有3.5个字符周期的空闲时间否则就会出现粘包的情况。针对3.5个字符周期，其实是一个具体时间，但是这个时间跟波特率相关。

2. Modbus-ASCII

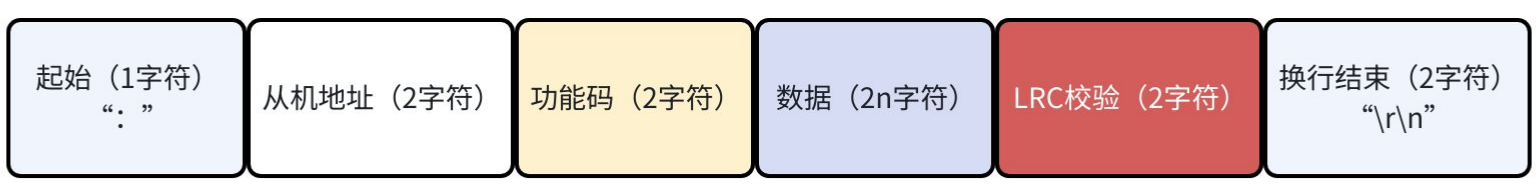
对于RTU协议，比如RTU发送一个字节： `0x12` ；ASCII协议则需要发送2个字节：一个字节代表ASCII码 `1` ，一个代表ASCII码 `2` ，即 `0x31` 和 `0x32` ，才能代表 `0x12` 。所以，ASCII码发送数据量是RTU方式的2倍，ASCII协议的效率比较低。

那么为什么还要这种模式?

假如你要发送数据 `0x03` ，采用RTU方式(16进制发送)，计算机中端设备接收到0x03后是不可以显示的，就是不能把0x03打印出来。因为可见字符的ASCII码是从 `32–126` ，不是这个范围以外的显示屏上都看不到，会出现乱码，如果是串口助手的话就会显示 `□□□□` 。如果采用ASCII方式(文本模式发送)，就不会出现不可显示和乱码的情况，因为文本模式发送0x03，就是发送ASCII码0和ASCII码3，也就是 `0x30` 和 `0x33` ，是可以正常显示在计算机中端的。

帧结构如下图所示：每个字节编码为 2 个 ASCII HEX 字符

帧结构 = 起始字符 “:” + 从机地址 (2 字符) + 功能码 (2 字符) + 数据 (2n 字符) + LRC 校验 (2 字符) + 换行



3. Modbus-TCP

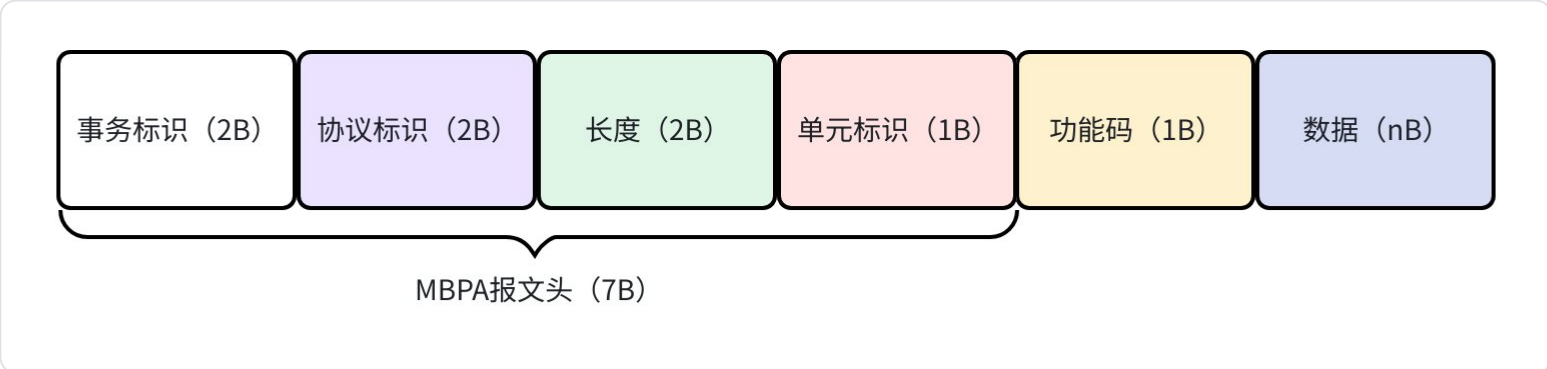
Modbus-TCP（也称 Modbus TCP/IP）是将 Modbus 应用层协议封装在 TCP/IP 网络传输层之上的协议版本。

主要特点包括：

- 使用 TCP 502 端口
- 通过 MBAP Header 替代 RTU 帧头与 CRC
- 速度快、无时序限制，可跨网段传输
- 功能码与寄存器模型与 Modbus RTU 完全一致
- 可支持多个客户端同时访问同一个从站设备

帧结构如下图所示：采用二进制编码

帧结构 = MBAP 头 (7B，事务 ID (2B) + 协议 ID (2B, Modbus固定为0) + 长度 (2B，表示后续 单元ID+PDU 的总长度) + 单元 ID (1B = 从站地址)) + 功能码 (1B) + 数据 (nB)



功能码

Modbus 功能码用于告诉从站设备要执行何种操作，是所有 Modbus 通讯中必不可少的参数，是所有 Modbus 指令的基础。下表展示了常见的Modbus功能码：

功能码（Hex）	名称	作用
0x01	读线圈状态	读取一组开关量输出
0x02	读离散输入	读取一组开关量输入
0x03	读保持寄存器	读取一个或多个保持寄存器
0x04	读输入寄存器	读取一个或多个输入寄存器
0x05	写单个线圈	控制单个开关量输出
0x06	写单个保持寄存器	设置单个保持寄存器值
0x0F(15)	写多个线圈	批量控制开关量输出
0x10(16)	写多个保持寄存器	批量设置保持寄存器

异常响应

Modbus 异常响应是从站在无法正确处理主站请求时返回的错误报文，属于协议规定的标准报错机制。此时：

功能码：原功能码 + 0x80（最高位 1）。

数据域：1 字节异常码，常见如下：

- 01：非法功能码
- 02：非法数据地址
- 03：非法数据值
- 04：从站设备故障

栗子☒🌰

RTU

1. 查询功能码0x03：

主机发送: `01 03 00 00 00 01 84 0A`

`01` -从机地址

`03` -功能码，读保持寄存器

`00 00` -起始寄存器地址，从0x0000开始查询，一个寄存器地址对应2字节的数据

`00 01` -要读取的寄存器数量，1个

`84 0A` -CRC校验码（实际 CRC 值是 `0x0A84` （低字节 `84` 在前，高字节 `0A` 在后））

从机回复: `01 03 02 19 98 B2 7E`

`01` -从机地址

`03` -功能码，读保持寄存器（正常响应）

`02` -后面跟着2个字节的数据

`19 98` -真正的数据内容

`B2 7E` -CRC校验码

2. 修改功能码0x06：

主机发送: `01 06 00 00 00 01 48 0A`

`01` -从机地址

`06` -功能码，写单个保持寄存器

`00 00` -要写入的起始寄存器地址，从0x0000开始修改

`00 01` -要修改的值，即修改0号寄存器值为00 01

`48 0A` -CRC校验码

从机回复: `01 06 00 00 00 01 48 0A`

这里如果回复的一样，说明这个数据是修改成功的；如果功能码不是06，而是别的，说明从机回复的数据有误，主机可以做相应的处理。

3. 写多寄存器0x10（16）：

主机发送： `01 10 00 00 00 02 04 00 01 00 02 C4 B2`

`01` -从机地址

`10` -功能码，写多个保持寄存器

`00 00` -起始寄存器地址，从0x0000开始写

`00 02` -要写入的寄存器数量，2个

`04` -后续数据的字节数

`00 01` -第一个寄存器的值

`00 02` -第二个寄存器的值

`C4 B2` -CRC校验码

从机回复： `01 10 00 00 00 02 0B CC`

`01` -从机地址

`10` -功能码，写多个保持寄存器

`00 00` -起始寄存器地址，确认写的位置没问题

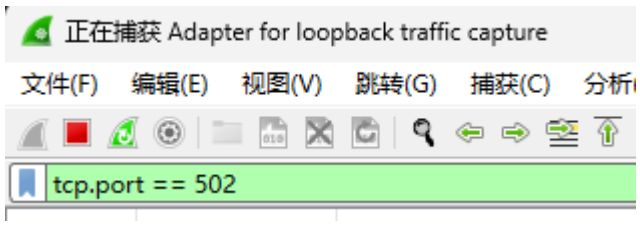
`00 02` -成功写了2个寄存器

`0B CC` -CRC校验码

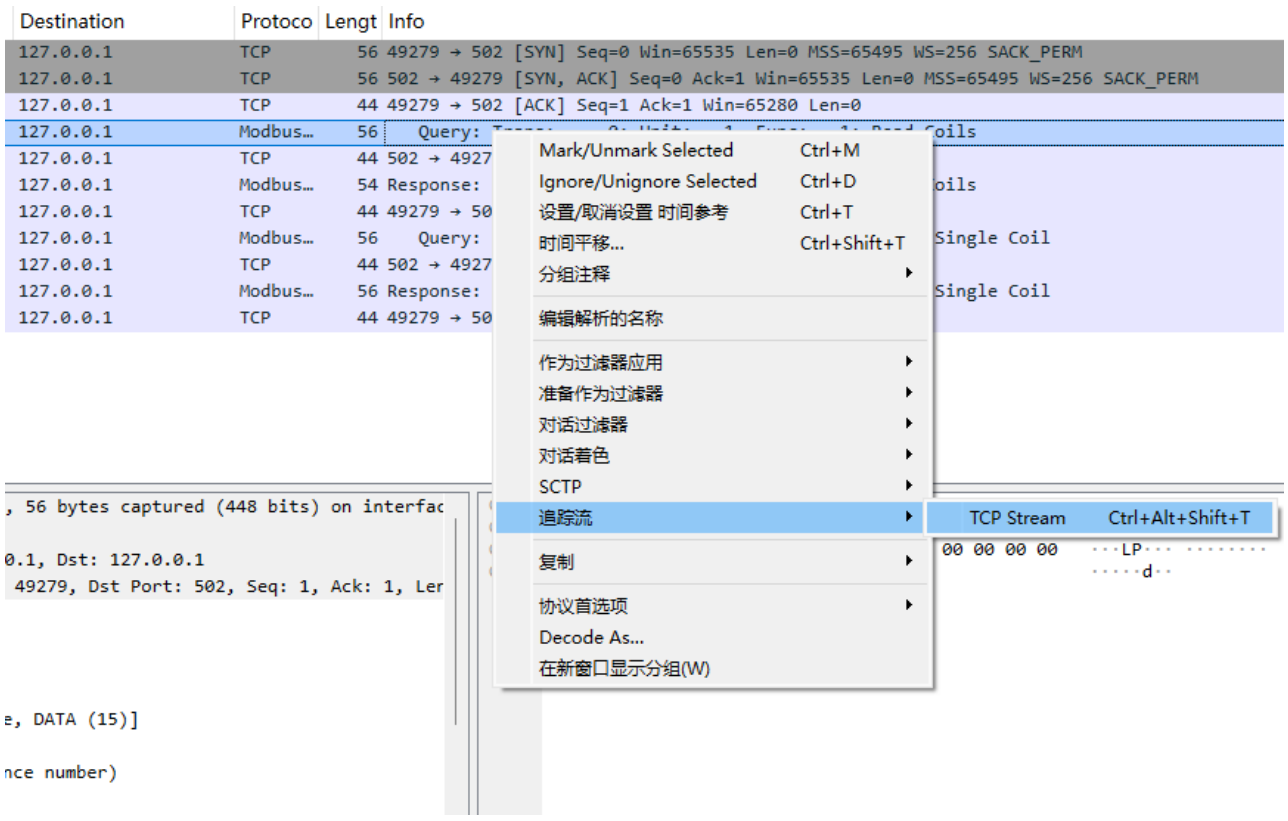
TCP

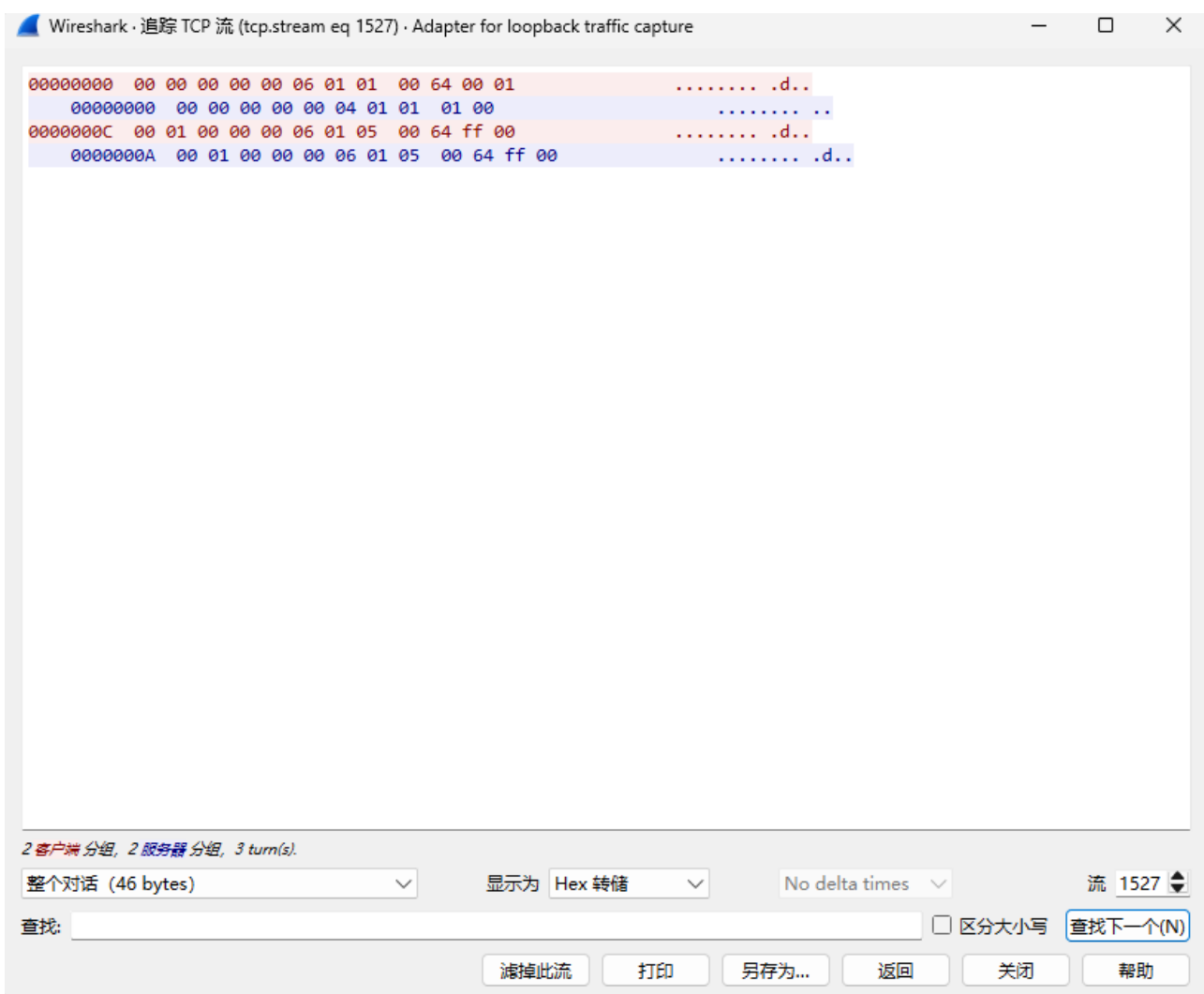
用Hsl和Wireshark进行抓包试验

1. 在Wireshark中选择对应的网卡并添加过滤

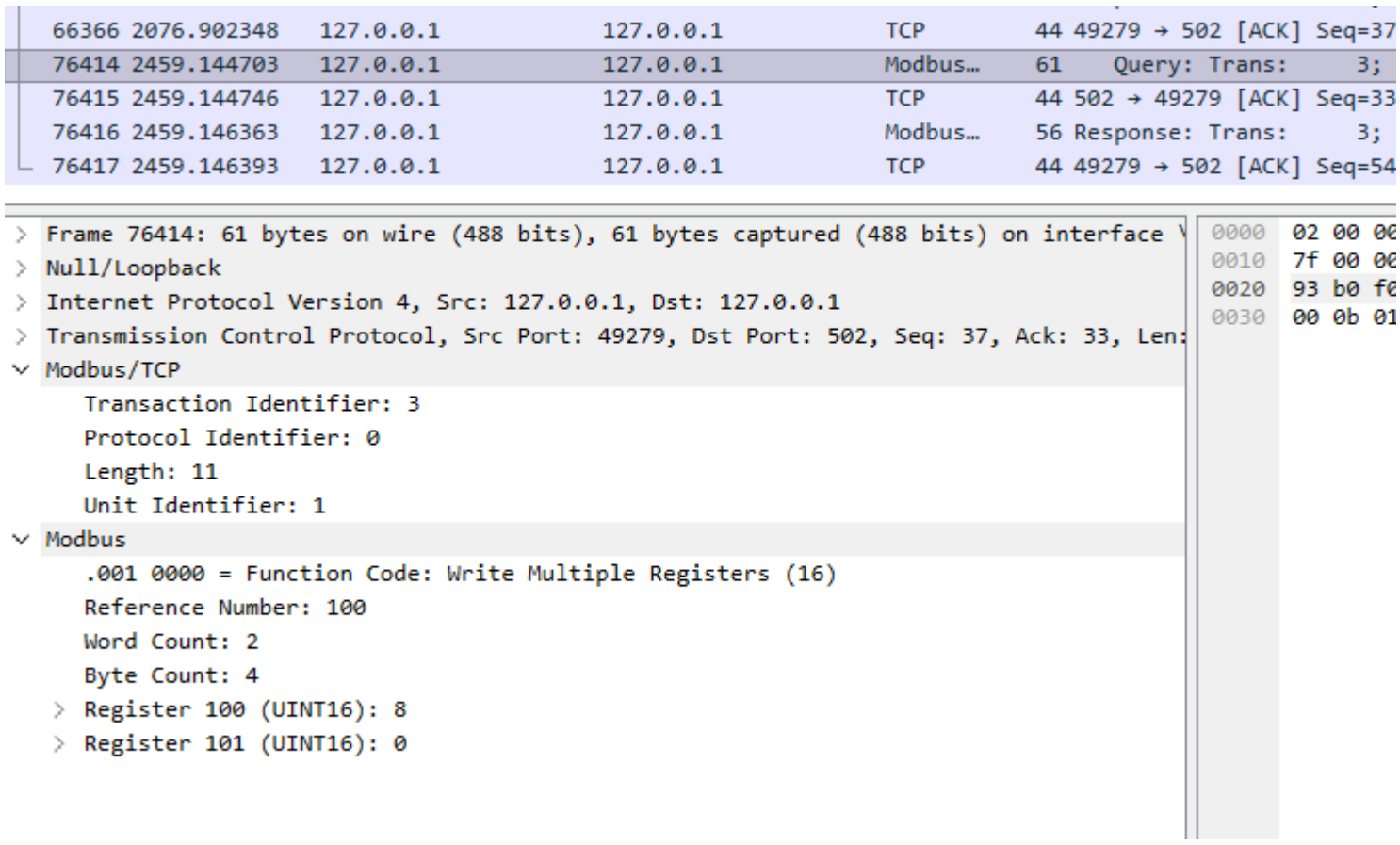


2. 在Hsl中启动Modbus虚拟服务器和Tcp连接
3. 可以尝试进行bool写入和读取等操作，可以在Wireshark中看到抓到的包
4. 右键数据包选择“追踪流-TCP Stream”，可以方便看到客户机与服务器的对话（下面显示为十六进制）





5. 该窗口下点击一行数据，即会在主窗口高亮对应的数据包，左下角展开即可看到各位数据的含义



内容来源

https://developer.aliyun.com/article/1421663

https://blog.csdn.net/tiandiren111/article/details/118347661

https://zhuanlan.zhihu.com/p/145546574

https://www.xlink.cn/docs/platform/protocol-description/Modbus/code/